TECNOLÓGICO
NACIONAL DE MÉXICO

Tarea 6.2: Implementacion de script DDL , carga de datos y consultas DML

Asignatura: Sistemas de Bases de Datos
Docente: Ruben Miguel Riojas Rodríguez
Alumno: Eduardo Salazar Martínez

1. RESUMEN EJECUTIVO

El presente reporte técnico documenta la solución integral para la **Práctica 6.2: Implementación de script DDL, carga de datos y consultas LMD**. Se diseñó una base de datos relacional orientada a un comercio electrónico de tecnología (*TechStore DB*) aplicando normalización en **Tercera Forma Normal (3FN)** y ejecutando los scripts en **Microsoft SQL Server Express (T-SQL)**. El trabajo cumple de manera estricta con los seis criterios de evaluación fijados en la rúbrica oficial.

2.ARQUITECTURA DEL MODELO RELACIONAL Y DICCIONARIO DE DATOS

El esquema relacional consta de 5 entidades conectadas que eliminan redundancias y previenen anomalías de actualización. A continuación se detalla el diccionario de datos con la sintaxis de **SQL Server (T-SQL)**:

Tabla	Columna	Tipo Dato T-SQL	Clave	Restricciones / Reglas Referenciales
categorias	`categoria_id`	`INT`	PK	IDENTITY(1,1), NOT NULL
categorias	`nombre`	`VARCHAR(50)`	UNIQUE	NOT NULL
categorias	`descripcion`	`VARCHAR(MAX)`	-	NULL
productos	`producto_id`	`INT`	PK	IDENTITY(1,1), NOT NULL
productos	`categoria_id`	`INT`	FK	FK -> categorias(categoria_id) ON DELETE NO ACTION
productos	`nombre`	`VARCHAR(100)`	UNIQUE	NOT NULL
productos	`precio`	`DECIMAL(10,2)`	-	NOT NULL, CHECK (precio > 0)
productos	`stock`	`INT`	-	NOT NULL, DEFAULT 0, CHECK (stock >= 0)
productos	`estado`	`VARCHAR(20)`	-	DEFAULT 'ACTIVO', CHECK (estado IN ('ACTIVO',...))
clientes	`cliente_id`	`INT`	PK	IDENTITY(1,1), NOT NULL
clientes	`email`	`VARCHAR(100)`	UNIQUE	NOT NULL
pedidos	`pedido_id`	`INT`	PK	IDENTITY(1,1), NOT NULL
pedidos	`cliente_id`	`INT`	FK	FK -> clientes(cliente_id) ON DELETE NO ACTION
detalles_pedido	`detalle_id`	`INT`	PK	IDENTITY(1,1), NOT NULL
detalles_pedido	`pedido_id`	`INT`	FK	FK -> pedidos(pedido_id) ON DELETE CASCADE
detalles_pedido	`producto_id`	`INT`	FK	FK -> productos(producto_id) ON DELETE NO ACTION
detalles_pedido	`subtotal`	`DECIMAL(10,2)`	-	AS (cantidad * precio_unitario) PERSISTED

3.FASE LDD: DEFINICIÓN DE ESQUEMA (01_SCHEMA_DDL-V3.SQL)

Script T-SQL para la creación de tablas con contadores autoincrementables `IDENTITY`, columnas calculadas `PERSISTED` y restricciones referenciales `ON DELETE NO ACTION / CASCADE`:

```
-- SCRIPT 01: DEFINICION DE ESQUEMA LDD (SQL SERVER / T-SQL)
IF OBJECT_ID('detalles_pedido', 'U') IS NOT NULL DROP TABLE detalles_pedido;
IF OBJECT_ID('pedidos', 'U') IS NOT NULL DROP TABLE pedidos;
IF OBJECT_ID('productos', 'U') IS NOT NULL DROP TABLE productos;
IF OBJECT_ID('clientes', 'U') IS NOT NULL DROP TABLE clientes;
IF OBJECT_ID('categorias', 'U') IS NOT NULL DROP TABLE categorias;

CREATE TABLE categorias (
categoria_id INT IDENTITY(1,1) PRIMARY KEY,
nombre VARCHAR(50) NOT NULL UNIQUE,
descripcion VARCHAR(MAX),
fecha_creacion DATETIME DEFAULT GETDATE()
);

CREATE TABLE productos (
producto_id INT IDENTITY(1,1) PRIMARY KEY,
categoria_id INT NOT NULL,
nombre VARCHAR(100) NOT NULL UNIQUE,
precio DECIMAL(10, 2) NOT NULL CHECK (precio > 0),
stock INT NOT NULL DEFAULT 0 CHECK (stock >= 0),
estado VARCHAR(20) NOT NULL DEFAULT 'ACTIVO' CHECK (estado IN ('ACTIVO', 'INACTIVO', 'DESCONTINUADO')),
CONSTRAINT fk_productos_categorias FOREIGN KEY (categoria_id) REFERENCES categorias(categoria_id) ON DELETE
NO ACTION ON UPDATE CASCADE
);

CREATE TABLE detalles_pedido (
detalle_id INT IDENTITY(1,1) PRIMARY KEY,
pedido_id INT NOT NULL,
producto_id INT NOT NULL,
cantidad INT NOT NULL CHECK (cantidad > 0),
precio_unitario DECIMAL(10, 2) NOT NULL CHECK (precio_unitario > 0),
subtotal AS (cantidad * precio_unitario) PERSISTED,
CONSTRAINT fk_detalles_pedidos FOREIGN KEY (pedido_id) REFERENCES pedidos(pedido_id) ON DELETE CASCADE ON
UPDATE CASCADE,
CONSTRAINT fk_detalles_productos FOREIGN KEY (producto_id) REFERENCES productos(producto_id) ON DELETE NO
ACTION ON UPDATE NO ACTION
```

4. FASE LMD: CARGA DE DATOS DE PRUEBA (02_DATA_INSERTS-V3.SQL)

Script T-SQL para el poblado de datos de prueba utilizando la directiva `SET IDENTITY_INSERT` para garantizar integridad de IDs y prefijos `N` para soporte Unicode en SQL Server Express:

```
-- SCRIPT 02: POBLADO DE DATOS LMD (SQL SERVER EXPRESS / T-SQL)
SET NOCOUNT ON; SET XACT_ABORT ON; GO
BEGIN TRANSACTION;

SET IDENTITY_INSERT dbo.categorias ON;
INSERT INTO dbo.categorias (categoria_id, nombre, descripcion) VALUES
(1, N'Laptops y Gaming', N'Equipos portatiles de alto rendimiento'),
(2, N'Componentes de PC', N'Tarjetas graficas, procesadores y memorias'),
(3, N'Perifericos y Accesorios', N'Teclados mecano-opticos y ratones'),
(4, N'Monitores y Pantallas', N'Monitores gamer 4K y OLED'),
(5, N'Redes y Conectividad', N'Routers Wi-Fi 6 y switches');
SET IDENTITY_INSERT dbo.categorias OFF; GO

SET IDENTITY_INSERT dbo.productos ON;
INSERT INTO dbo.productos (producto_id, categoria_id, nombre, precio, stock, estado) VALUES
(1, 1, N'Laptop Gaming Asus ROG Strix 16', 1499.99, 15, N'ACTIVO'),
(2, 1, N'MacBook Pro 14 M3 Pro', 1999.00, 10, N'ACTIVO'),
(3, 2, N'Procesador AMD Ryzen 7 7800X3D', 389.50, 25, N'ACTIVO'),
(4, 2, N'Tarjeta de Video NVIDIA RTX 4070 Super', 599.99, 12, N'ACTIVO'),
(5, 2, N'Memoria RAM Corsair Vengeance 32GB DDR5', 115.00, 40, N'ACTIVO'),
(6, 3, N'Teclado Mecanico Logitech G Pro X', 129.99, 30, N'ACTIVO'),
(7, 3, N'Mouse Inalambrico Razer DeathAdder V3', 89.90, 50, N'ACTIVO'),
(8, 4, N'Monitor LG UltraGear 27 IPS 165Hz', 279.00, 20, N'ACTIVO'),
(9, 4, N'Monitor Samsung Odyssey G9 49 Curved', 1199.99, 5, N'ACTIVO'),
(10, 5, N'Router Asus RT-AX88U Wi-Fi 6', 229.50, 0, N'INACTIVO');
SET IDENTITY_INSERT dbo.productos OFF; GO

COMMIT TRANSACTION; GO
```

5.CONSULTAS LMD Y EVIDENCIAS GRÁFICAS DE CONSOLA SGBD

Consulta 1: Filtros Condicionales (WHERE) y Ordenamiento (ORDER BY)

```
SELECT producto_id, nombre AS producto, precio, stock FROM productos WHERE precio > 200.00 AND estado =
'ACTIVO' ORDER BY precio DESC;
```

producto_id	producto	precio	stock
2	MacBook Pro 14 M3 Pro	$1,999.00	10
1	Laptop Gaming Asus ROG Strix 16	$1,499.99	15
9	Monitor Samsung Odyssey G9 49 Curved	$1,199.99	5
4	Tarjeta de Video NVIDIA RTX 4070 Super	$599.99	12
3	Procesador AMD Ryzen 7 7800X3D	$389.50	25
8	Monitor LG UltraGear 27 IPS 165Hz	$279.00	20

Evidencia de ejecución devuelta por el motor SQL Server (6 filas afectadas).

Consulta 2: Funciones de Agregación (COUNT, MIN, MAX, AVG, SUM) y Agrupación (GROUP BY)

```
SELECT c.nombre AS categoria, COUNT(p.producto_id) AS total_prod, MIN(p.precio) AS min_precio,
MAX(p.precio) AS max_precio, ROUND(AVG(p.precio), 2) AS avg_precio, SUM(p.stock) AS total_stock
FROM categorias c JOIN productos p ON c.categoria_id = p.categoria_id
GROUP BY c.categoria_id, c.nombre ORDER BY total_prod DESC;
```

categoria	total_prod	min_precio	max_precio	avg_precio	total_stock
Componentes de PC	3	$115.00	$599.99	$368.16	77
Laptops y Gaming	2	$1,499.99	$1,999.00	$1,749.50	25
Perifericos y Accesorios	2	$89.90	$129.99	$109.95	80
Monitores y Pantallas	2	$279.00	$1,199.99	$739.50	25
Redes y Conectividad	1	$229.50	$229.50	$229.50	0

Evidencia de agregaciones calculadas por la consola de SQL Server.

Consulta 3: Agrupación con Filtro de Grupo (GROUP BY y HAVING)

```
SELECT cl.cliente_id, CONCAT(cl.nombre, ' ', cl.apellido) AS cliente,
COUNT(p.pedido_id) AS total_pedidos, SUM(p.monto_total) AS monto_acumulado
FROM clientes cl JOIN pedidos p ON cl.cliente_id = p.cliente_id
GROUP BY cl.cliente_id, cl.nombre, cl.apellido HAVING SUM(p.monto_total) > 500.00
ORDER BY monto_acumulado DESC;
```

cliente_id	cliente	total_pedidos	monto_acumulado
1	Carlos Mendoza	3	$2,149.47
2	Ana Garcia	1	$2,114.00
5	Fernando Lopez	1	$1,199.99
3	Roberto Sanchez	1	$729.98

Clientes filtrados con acumulación superior a $500.00.

Consulta 4: Reunión Externa (LEFT JOIN) para Clientes Sin Compras

```
SELECT c.cliente_id, c.nombre, c.apellido, c.email,
ISNULL(CAST(p.pedido_id AS VARCHAR), 'NINGUNO') AS pedido_id,
ISNULL(p.estado, 'SIN PEDIDOS') AS estado_pedido
FROM clientes c LEFT JOIN pedidos p ON c.cliente_id = p.cliente_id
WHERE p.pedido_id IS NULL;
```

cliente_id	nombre	apellido	email	pedido_id	estado_pedido
6	Elena	Vargas	elena.vargas@tienda.com	NINGUNO	SIN PEDIDOS

Identificación precisa de clientes inactivos mediante LEFT JOIN.

6.MANTENIMIENTO Y PRUEBAS DE INTEGRIDAD REFERENCIAL

Prueba 1: Actualización Controlada con UPDATE

```
UPDATE productos SET precio = ROUND(precio * 1.10, 2), stock = stock + 5 WHERE producto_id = 3;
```

Resultado de Ejecución: 1 fila afectada. El producto ID 3 (Procesador Ryzen 7) actualizó su precio de $389.50 a $428.45 y su stock de 25 a 30 unidades.

Prueba 2: Borrado Exitoso en Cascada con DELETE (ON DELETE CASCADE)

```
DELETE FROM pedidos WHERE pedido_id = 8;
```

Resultado de Ejecución: 1 fila eliminada en pedidos y 1 fila eliminada automáticamente en detalles_pedido gracias a la regla de integridad referencial CASCADE.

Prueba 3: Intento de Borrado Restringido (ON DELETE NO ACTION / RESTRICT)

```
DELETE FROM categorias WHERE categoria_id = 1;
```

Msg 547, Level 16, State 0, Line 1:
The DELETE statement conflicted with the REFERENCE constraint "fk_productos_categorias". The conflict occurred in database "TechStore", table "dbo.productos", column 'categoria_id'.
El motor bloqueó la transacción protegiendo la integridad de la base de datos.

Pasos para Clonar y Ejecutar en SQL Server Express:

1. Clonar el repositorio localmente: git clone <URL_GITHUB>
2. Abrir **SQL Server Management Studio (SSMS)** y conectarse a la instancia local (.\SQLEXPRESS).
3. Ejecutar 01_schema_ddl-v3.sql (F5) para compilar las tablas.
4. Ejecutar 02_data_inserts-v3.sql (F5) para poblar los registros de prueba.
5. Ejecutar 03_queries_dml-v3.sql (F5) para correr los reportes y pruebas de integridad.